

FULL-STACK FRONTEND TRACK · WEEK 2 · DAY 5 · PART 1

Lab Task — Part 1

Arrays, Destructuring & Object Reshaping — building an Order Analytics Dashboard

Hands-on Practice

Est. 4+ hours

Contents

- 1. Overview & Instructions
- 2. Warmups
- 3. Applied Project — Order Analytics Dashboard
- 4. Self-Check Checklist

1. Overview & Instructions

This is Part 1 of today's lab, covering everything up through Section 4 of the lecture. Part 2 picks up from there and continues separately.

Rules for This Task

Do

Work with plain objects and arrays — destructure them, spread them, chain array methods on them. Prefer built-in array methods over manual loops wherever one fits.

Don't

No `class`, no `import / export`, no separate module files — none of that has been introduced yet at this point in the day. Everything today lives in one plain script. No API calls.

Deliverables

File	Purpose
warmups.js	Section 2
orders.html + orders.js	Section 3, the applied project

2. Warmups

warmups.js

More ground to cover than a typical warmup set — work through all of these before moving to the applied project.

2.1 — Arrays: the Basics

- Build an array of at least 6 city names. Using `push` / `pop` / `shift` / `unshift` , perform one of each operation, logging the array after every single one so the effect of each is visible
- Log `.length` before and after adding a duplicate entry, confirming duplicates are allowed and counted
- Loop over the array with a classic `for` loop, then again with `for...of` — both should print identical output

2.2 — Destructuring

- Given `const order = { id: 101, customer: "Sara Ahmed", total: 249.99, status: "Shipped" };` , destructure `customer` and `total` in one line
- Destructure the first two elements out of an array of at least 4 numbers, and separately capture "everything else" using rest syntax in the same line
- Write a function that takes an order object and destructures it directly in the parameter list, returning a one-line summary string

2.3 — Spread & Rest

- Given two separate arrays of order ids (e.g. online orders, in-store orders), combine them into one array using spread — don't use `concat`
- Given an order object, create an updated copy with its `status` changed to `"Cancelled"` using object spread, then log both the original and the copy to confirm the original's status didn't change
- Write a function using a rest parameter that accepts any number of order totals and returns them as an array (no summing yet — that's 2.4)

2.4 — Array Methods

Requirement

Given an array of at least 8 order objects (each with `id` , `customer` , `total` , `status`), without writing a single manual loop:

- Use `reduce` to get the combined total of all orders
- Use `filter` to get only orders with `status === "Shipped"`
- Use `map` to produce an array of just the customer names
- Use `find` to get the first order over \$200
- Use `some` and `every` to check, respectively, whether any order is `"Cancelled"` and whether all orders have a total greater than 0
- Use `sort` with a compare function to sort the orders by `total` , highest first
- Chain at least one `filter` directly into a `map` in a single statement

3. Applied Project — Order Analytics Dashboard

orders.html + orders.js

A single Bootstrap-styled page that turns a local dataset of orders into a small analytics view. No classes, no modules, no API — every requirement below is meant to be solved with the tools from Sections 1–4 of the lecture.

1. Local Data

- A `const orders` array of at least 15 order objects, each with: `id`, `customer`, `status` (one of `"Pending"` / `"Shipped"` / `"Cancelled"`), `date`, and `items` — an array of at least 1–3 line items per order, each item having `product`, `price`, and `quantity`

2. Per-Order Totals

Requirement

Write a function that takes one order and returns its total value (sum of `price × quantity` across its `items`) using `reduce`. This function gets reused by nearly everything below — write it once, call it everywhere you need an order's value.

3. Dashboard Summary Numbers

- Total revenue across every order (combine your per-order totals)
- Number of orders per status — `Pending` / `Shipped` / `Cancelled` counts, computed with array methods, not manual counters
- The single highest-value order (customer name and amount)
- Whether every order has at least one item (a sanity check using `every`)

4. Top Customers

- A list of the 3 highest-value orders, sorted highest to lowest, each showing customer name and total
- A list of unique customer names who have at least one order over \$150 — no duplicate names, even if a customer appears in multiple qualifying orders

5. Filtering & Search

- Buttons or a dropdown to filter the visible order list by status
- A text input that filters the visible order list by customer name as the user types (partial, case-insensitive match)

6. Immutable Discount Preview

Requirement

Add a "Preview 15% Off Everything" toggle. When enabled, build a completely new array of orders where every item's `price` is reduced by 15% — using spread to copy each order and each item, never mutating the original `orders` array or its nested `items` arrays. Recompute and display the discounted totals from this new array. Turning the toggle off must restore the exact original numbers, proving nothing was mutated.

The part people get wrong

Spreading `{ ...order }` only copies the order's top-level fields — its `items` array inside is still the *same* array reference as the original. You need to spread the items array (and each item inside it) too, or changing a discounted item's price will silently change the original order's price as well.

7. Rendering

- Render the (possibly filtered, possibly discounted) order list as Bootstrap cards, each showing customer, status, item count, and total
- Use destructuring in your rendering function's parameters wherever it makes the code cleaner

4. Self-Check Checklist

	Requirement
☐	No manual loops anywhere the task asked for an array method instead
☐	The per-order total function is written once and reused everywhere a total is needed
☐	All four dashboard summary numbers are correct and computed with array methods
☐	Top-3 list is genuinely sorted, not just the first 3 items in data order
☐	The unique-customers list has no duplicate names
☐	Status filter and name search both work and can be combined
☐	The discount preview never mutates the original <code>orders</code> array — verified by toggling it off and confirming every number returns exactly to its original value
☐	Nested <code>items</code> arrays are spread-copied too, not just the outer order object
☐	At least one destructured function parameter appears somewhere in <code>orders.js</code>